

Independent Extraction Audit: Dungeon Drifters MicroPython Portability Work

Executive conclusion

The portability campaign produced **one genuinely strong extraction candidate, one meaningful upstream contribution area, and several useful engineering patterns that should not become products.**

The strongest reusable artifact is the **ahead-of-time dataclass field metadata architecture:**

- ordinary CPython dataclass source
- host-side discovery under full CPython semantics
- deterministic generated field manifest
- very small constrained-runtime consumer

That architecture addresses a real MicroPython problem rather than merely wrapping a missing function. Current MicroPython deliberately reduces runtime handling of annotations, while the current `micropython-lib` tree has no merged `dataclasses` package. ¹ A contemporary third-party implementation, `udataclasses`, is real and usable, but it solves the problem differently: it treats public class variables as fields, requires initial values for fields, sorts fields rather than preserving CPython source declaration order, and ignores types at runtime. That makes it a poor substitute for projects that want to keep normal CPython dataclass declarations such as annotation-only required fields.

Dungeon Drifters' solution therefore occupies a distinct niche: **preserve richer host-language source semantics without requiring the constrained interpreter to retain or reconstruct that semantic information at runtime.** The idea is broader than dataclasses, although the first package should emphatically remain about dataclasses rather than becoming a universal code-generation framework.

The recommendation is:

EXTRACT AFTER A SMALL GENERALIZATION PASS for the generated dataclass metadata system and its narrow runtime consumer.

I would implement it as **two separable components in one repository initially**, rather than immediately creating two separately governed projects:

```
host package / CLI
  discovers CPython dataclasses
  validates supported semantics
```

```
emits versioned deterministic metadata
```

```
runtime package
```

```
consumes that metadata
```

```
implements only the explicitly supported dataclass subset
```

```
has no host-side inspection/code-generation dependencies
```

The current DD implementation already has more extraction maturity than a historical description might suggest. The generator sorts its output deterministically and provides a `--check` mode that regenerates and compares the artifact, so **deterministic generation and basic stale-manifest detection already exist**. What it does not yet have is a public manifest contract, schema/version identifiers, robust import-side-effect policy, formally specified identity rules, package-independent discovery, or a tested general-purpose compatibility surface.

The second important conclusion concerns `StrEnum`: **do not extract DD's implementation**. DD currently installs a wrapper around `builtins.__build_class__` at import time and therefore intercepts class construction process-wide so it can recognize direct subclasses of its `StrEnum` base. That is an ingenious application-local workaround, but it is the wrong risk profile for a public compatibility library. More importantly, the ecosystem is moving toward the same feature through a much cleaner route: `micropython-lib` PR #980 is an open draft implementation of `Enum`, `IntEnum`, and `StrEnum`, and was still being updated in August 2026. The right destination is therefore **CONTRIBUTE UPSTREAM**, while DD keeps its existing workaround internal until a suitable upstream implementation is usable.

The third conclusion is deliberately negative. Most of MPY0-MPY20 should **not** become packages. `first_or_none`, removal of `Counter`, removal of `groupby`, removal of strict `zip`, starred-tuple rewrites, ASCII validation changes, the random adapter, regular-expression `fullmatch` emulation, readonly mapping, no-op typing constructs, and similar changes are good portability engineering but weak product ideas. Several are best understood as **source simplification**, which is often better than carrying a compatibility abstraction indefinitely.

The qualification system is also valuable but not yet a product. DD's semantic-signature comparison, forced-overlay execution, raw-runtime probe, fresh-process heap sweep, and qualification matrix form an unusually good **testing pattern**. The current orchestrator, however, knows DD's stage names, tests, probe output protocol, MicroPython build policy, and semantic signature marker. The heap controller similarly knows DD's probe protocol and failure markers. I would document and reuse the pattern, but I would not publish a general framework until a second unrelated project demonstrates what the genuinely common interface is.

The audit basis is the live `mpy` branch, not the historical proposal documents. The inspected branch was at commit `e31c77e3ccd0628130286d722a270d98deaf4bed`, and the live tree contains the portability implementations, generator, qualification tools, and associated tests discussed below.

Bottom line: DD did not accidentally produce twenty libraries. It appears to have produced **one credible new toolchain idea**, a handful of plausible `micropython-lib` contributions, and many project-specific compatibility decisions that should remain exactly that.

Extraction map

The destination codes below use the choices in the audit brief. “Maturity” means maturity as a reusable artifact, not whether the mechanism works for DD.

Mechanism	Decision	Destination	Target user and problem	Existing overlap	Maturity
Generated dataclass metadata	EXTRACT AFTER A SMALL GENERALIZATION PASS	B	Developers keeping normal CPython dataclass source while deploying to MicroPython or another constrained Python runtime that cannot cheaply recover field declarations	<code>udataclasses</code> solves a related but semantically different problem; no merged <code>micropython-lib</code> dataclasses package was found	High pro Current CPython determin <code>qualnam</code> <code>names</code> ; changed public so versionin
MicroPython dataclass compatibility overlay	EXTRACT AFTER A SMALL GENERALIZATION PASS , but only as the runtime half of the metadata system	B	Same users; turns manifest data into useful constructor/equality/frozen behavior without runtime annotation reflection	<code>udataclasses</code> is the main current third-party comparison	Producti <code>datacl</code> currently <code>frozen</code> <code>default</code> <code>__init</code> <code>__post</code> impleme
Dataclass field reconstruction	EXTRACT WITH THE ABOVE , not separately	B	Reconstruct ordered field behavior from generated names plus class-level defaults/factory markers	Integral to the preceding architecture	Useful re independe
Readonly mapping compatibility	KEEP INTERNAL	F	DD needs a read-only view where <code>MappingProxyType</code> is unsuitable/unavailable	Many trivial wrappers can provide the same three-method interface	Current <code>__geti</code> length o mapping distinctiv package
StrEnum compatibility	CONTRIBUTE UPSTREAM , do not publish DD implementation	C	MicroPython users wanting normal Enum/IntEnum/StrEnum behavior	Active <code>micropython-lib</code> PR #980 already implements all three.	DD beha own clas impleme public-lik

Mechanism	Decision	Destination	Target user and problem	Existing overlap	Maturity
Enum value reconstruction / lookup	DO NOT EXTRACT from DD	C/G	Same Enum users	Proper Enum implementation should own this behavior	DD replacement attribute <code>.name / __members__ / __new__</code> Useful as not as a
Keyword compatibility	CONTRIBUTE UPSTREAM if parser-compatible tests confirm semantics	C	Code expecting <code>keyword.iskeyword()</code> / <code>issoftkeyword()</code> on MicroPython	Current <code>micropython-lib</code> master inspection did not find a <code>python-stdlib/keyword</code> package.	DD implementation sets and a reduced a standard
Regex <code>fullmatch</code> compatibility	KEEP INTERNAL	F	DD call sites needing whole-string matching	This is a one-function adaptation, not a product	Current <code>match()</code> boundary
<code>collections.abc</code> compatibility	KEEP INTERNAL	F	Narrow runtime checks for collection-like objects	General ABC semantics are much broader	DD fallback <code>__len__ / __contains__</code> as <code>collections.abc</code> compatibility semantic
Typing / Protocol compatibility	KEEP INTERNAL	F	Allow runtime imports of code whose typing objects are irrelevant at execution time	Static type checking is performed on the host; MicroPython deliberately reduces runtime annotation handling. ²	DD's <code>Pr</code> aliases a <code>runtime</code> intention substituting scaffolding <code>typing</code>
Portable ASCII/string validation	DO NOT EXTRACT	H	Source code that needs equivalent validation on both runtimes	Direct string logic	A source carrying

Mechanism	Decision	Destination	Target user and problem	Existing overlap	Maturity
Counter dependency removal	DO NOT EXTRACT	H	Code that can perform its counting directly	<code>Counter</code> itself is already the abstraction on full Python	Removing extracted
Starred tuple portability changes	DO NOT EXTRACT	H	Syntax/runtime portability	Source rewrite	No dual abstraction
Strict <code>zip</code> removal	DO NOT EXTRACT	H	Compatibility with runtimes lacking the CPython option	Explicit source validation or ordinary iteration	Again a s decision,
Session persistence import boundary	KEEP INTERNAL	E	DD needs gameplay/runtime code not to pull host-only persistence machinery into constrained execution	General dependency inversion / boundary design	Architect abstract the appli
<code>itertools.groupby</code> removal	DO NOT EXTRACT	H	Simplify code for reduced runtimes	Direct loop/state logic	No packa results
<code>first_or_none</code>	KEEP INTERNAL	F	Return the first iterable item or <code>None</code>	One ordinary <code>for</code> loop	The entire effective Packaging depende
Portable runtime random adapter	KEEP INTERNAL	F	Let DD accept an injected RNG while falling back from <code>randrange</code> to <code>getrandbits</code>	General random libraries already own broad RNG APIs	DD's reje clean and small ap
Probe/bootstrap infrastructure	KEEP INTERNAL; document pattern	E	DD needs to start the real runtime under controlled overlays and report machine-readable probe state	Test runners and subprocess tooling overlap with pieces, but not DD's exact protocol	Reusable shaped i
Cross-runtime qualification tooling	RESEARCH FURTHER , but do not extract yet	E initially	Potentially libraries that need semantic equivalence across CPython and MicroPython	Generic test runners do not by themselves define semantic equivalence	DD matr machine signature overlay multiple consume freezing

Mechanism	Decision	Destination	Target user and problem	Existing overlap	Maturity
Constrained-heap qualification tooling	RESEARCH FURTHER , but keep DD implementation internal	E initially	MicroPython projects wanting reproducible memory-floor qualification	Generic process/test tooling only partially overlaps	Fresh-pro structure valuable probe pa criteria a

There are therefore only **three areas that merit action outside DD**:

1. the dataclass metadata/toolchain, as a new narrowly scoped project;
2. Enum/StrEnum, through upstream participation rather than extracting DD's implementation;
3. possibly tiny stdlib-compatibility contributions such as `keyword`, where `micropython-lib` is a more appropriate home than a new package.

Everything else should either remain internal or disappear over time as source-level rewrites replace compatibility shims.

Dataclass metadata and runtime layer

The dataclass work is the strongest candidate because its value is not “DD needed `dataclass` on MicroPython.” That problem already has third-party solutions. Its distinctive value is **moving semantic discovery to the environment where the information exists**.

Current MicroPython documentation explicitly describes annotation handling as reduced compared with CPython and frames runtime retention/checking of annotations as an avoidable cost for constrained systems. ³ A MicroPython discussion around retaining class-variable type information illustrates precisely the kind of missing runtime metadata that complicates ordinary annotation-driven libraries.

DD's architecture sidesteps that limitation instead of emulating full Python reflection.

The current generator scans DD application modules, identifies classes decorated as dataclasses, imports those modules under CPython, asks actual CPython dataclass machinery which fields exist, and emits a Python module mapping `(module name, qualified class name)` to an ordered tuple of field names. The output is sorted deterministically. Its `--check` mode compares the generated content against the committed artifact and fails when they differ.

That distinction matters. Consider ordinary source conceptually equivalent to:

```
@dataclass
class Request:
    name: str
    retry_count: int = 0
```

On CPython, `name` is a field even though the class has no runtime value assigned to `name`. A runtime strategy that discovers fields by scanning public values in `Request.__dict__` cannot faithfully recover that declaration.

`udataclasses` deliberately chooses a different model. Its documentation says public class variables are treated as fields, fields require initial values, ordering differs from CPython dataclasses, and field types are not used at runtime. Its implementation discovers fields by traversing class dictionaries/MRO rather than relying on CPython's annotation-driven dataclass field model. It generates several methods dynamically using `eval` / `exec`.

That makes `udataclasses` a **complement, not a reason to abandon extraction**:

Approach	Source compatibility	Runtime metadata cost	Build step	Main tradeoff
Manual schemas	Low unless author duplicates declarations	Low	No	Two sources of truth
Runtime class-variable discovery such as <code>udataclasses</code>	Requires adopting its field conventions	Low/moderate	No	Source semantics differ from CPython dataclasses
Retaining annotations/reflection at runtime	High in principle	Potentially higher	No	Conflicts with constrained-runtime priorities and current MicroPython behavior
DD-style generated manifest	High for the supported dataclass subset	Low	Yes	Requires deterministic build integration
Fully generated model classes	Potentially high	Very low	Yes	Generated source can become the primary artifact and complicate debugging

The generated-manifest approach is particularly attractive to developers who already have a full CPython build/test environment and treat the MicroPython artifact as a deployment target. It pays complexity once at build time so the constrained runtime does not need `inspect`, annotation retention, AST processing, or rich `Field` objects.

What is already reusable

The best part of the current code is smaller than a full dataclass implementation.

The reusable core is:

```
CPython dataclass discovery
+
canonical model identity
+
ordered field extraction
+
deterministic artifact generation
+
artifact freshness check
```

The live generator already provides the last two properties.

The runtime side also has a coherent kernel. It looks up the class's `(module, __qualname__)` in the generated manifest, reconstructs each field's required/default/default-factory status from the class namespace, synthesizes initialization and equality behavior, invokes `__post_init__`, and applies its simple frozen behavior when requested.

The important insight is that **defaults do not necessarily belong in the manifest**. Keeping defaults and default factories in executable source avoids attempting to serialize arbitrary objects or callables. The manifest needs to preserve information that MicroPython loses, especially field identity and ordering. The source itself can remain responsible for executable behavior.

That should remain a design principle in the extracted project.

What is DD-specific and must be removed

The generator currently hardcodes DD's repository root, `src/app` discovery area, and output path. It parses DD source files to find bare `@dataclass` decorators, contains an application-motivated check for bare `input()` calls before importing candidate modules, imports the candidate modules, and writes directly into DD's portability tree.

None of those policies belongs in a public library.

A generalized host tool should instead accept explicit package/module roots:

```
mpy-dataclass-manifest build \
  --package mypackage \
  --output build/_dataclass_manifest.py
```

and:

```
mpy-dataclass-manifest check \  
  --package mypackage \  
  --manifest build/_dataclass_manifest.py
```

Those command names are conceptual, not proposed registered package names.

Discovery should support explicit module lists and package roots. Importing application code must be treated as **trusted-code execution**, not as safe static analysis. DD's special bare-`input()` detection is not a general import-safety mechanism and should be deleted rather than expanded into an endless blacklist.

An optional static discovery phase can narrow candidate modules before imports, but the authoritative discovery path should continue to use actual CPython dataclass semantics. Otherwise the project would gradually reimplement the CPython parser and decorator rules, defeating the architecture's advantage.

The manifest needs a real contract

Current DD identity is a tuple:

```
(module_name, qualified_name)
```

That is a good starting point.

A public artifact needs, at minimum:

```
manifest schema/version  
generator version  
identity mode  
model identity  
ordered field names
```

The schema should remain deliberately compact. A conceptual representation is:

```
MANIFEST_SCHEMA = 1  
IDENTITY_MODE = "module_qualname"  
  
MODELS = {  
    ("mypkg.models", "Request"): ("name", "retry_count"),  
}
```

Metadata that is useful only for diagnostics, such as fingerprints or generator provenance, should be separable so constrained deployments can strip it.

A **source fingerprint is useful but not mandatory for correctness**. DD's existing regenerate-and-compare check already detects every source change that changes the generated field artifact. Hashing entire source files would invalidate the artifact after unrelated edits and therefore create noise. A better public design would optionally fingerprint the **semantic generator input**, or retain full-file hashes only in build metadata that is not needed by the runtime.

Module and qualname rules need to be explicit:

```
top-level classes: supported
nested named classes: only if deliberately tested
local classes containing <locals>: reject
class aliases: identity remains defining module + qualname
module relocation: considered an identity change
dynamically synthesized anonymous classes: out of scope
```

Failing early is better than trying to make identity magical.

Runtime scope should be intentionally smaller than CPython dataclasses

The current DD module should not be published unchanged under a promise of “MicroPython dataclasses.”

Its `field()` recognizes only `default` and `default_factory`. The decorator essentially recognizes `frozen`; unsupported options raise errors. It synthesizes initialization and equality, supports `__post_init__`, and stores `__dataclass_fields__` as an ordered tuple of names rather than CPython's richer mapping of `Field` objects.

That last detail is especially important. A third-party library will reasonably interpret the presence of `__dataclass_fields__` as a compatibility promise. DD can get away with a much weaker representation because DD controls all consumers. A public package cannot silently make that assumption.

A first release should therefore explicitly support:

```
normal annotation-driven field discovery, via generated manifest
required constructor fields
default values
default_factory
declaration order
__init__
__eq__
__post_init__
frozen instances
```

It should either implement a normal dataclass-like `repr` or explicitly document that generated `repr` is not supported in the first release. The present DD code should not accidentally define the public standard simply because DD did not require a feature.

The first release should **deliberately reject**, rather than partially emulate:

```
order=True
unsafe_hash
slots
weakref_slot
kw_only
InitVar
ClassVar-sensitive runtime behavior unless proven
descriptor-field edge cases
field metadata APIs
dataclasses.fields() compatibility unless properly implemented
asdict / astuple / replace
dynamic make_dataclass
multiple dataclass inheritance
complex dataclass inheritance until explicitly tested
CPython-compatible Field reflection
pattern-matching integration beyond what MicroPython itself supports
```

Types of fields do **not** need to be present in the runtime manifest. Nested or imported field types are therefore not a special problem unless the project later decides to expose runtime type information. That would be a different product requirement and should not be smuggled into version one.

Host/runtime split

The cleanest product boundary is **two components, one project initially**.

A repository could look conceptually like:

```
micropython-dataclass-manifest/
  host/
    discovery.py
    schema.py
    generate.py
    cli.py

  runtime/
    dataclass_runtime.py
    field.py
    manifest.py
```

```
integration/  
    dataclasses.py.template  
  
tests/  
    fixtures/  
    cpython/  
    forced_runtime/  
    micropython/
```

Illustrative naming directions include:

```
mpy-dataclass-manifest  
microdataclass-aot  
dataclass-aot  
mpy-dc
```

Name availability has not been established here, so these are architectural labels rather than registry claims.

The **host component** belongs in the ordinary Python packaging ecosystem and may depend on the CPython standard library's `ast`, `dataclasses`, `importlib`, `inspect`, and filesystem APIs.

The **runtime component** should have **zero third-party dependencies** and should avoid importing the host component entirely.

The constrained runtime should not need `ast`, filesystem discovery, `inspect`, JSON parsing, or packaging metadata. A generated Python module is a good artifact because importing Python literals is already something MicroPython is optimized to do.

For unchanged application source that says:

```
from dataclasses import dataclass, field
```

the project should not install a process-wide import hook. Instead it should support a **MicroPython-only compatibility facade** named `dataclasses.py`, placed on the constrained deployment's import path. That facade can bind the generated manifest to the generic runtime implementation. CPython continues using its own standard library implementation.

This is essentially the clean part of DD's existing overlay architecture, separated from DD paths and policies.

`micropython-lib` itself is designed for reduced, compatible standard-library modules and supports installation through MicroPython package tooling such as `mip`; that makes it a plausible eventual home for the generic **runtime** part if its semantics mature enough. ⁴ The host generator, however, is inherently a CPython build tool and does not naturally belong there.

I would therefore **not submit the whole generated-manifest architecture to** `micropython-lib` **first**. Prove the external package contract independently. An eventual arrangement could be:

```
PyPI host generator
  ↓ emits
versioned manifest
  ↓ consumed by
tiny MicroPython runtime

later:
runtime compatibility subset may be suitable for micropython-lib
```

Test matrix before a public release

The DD campaign provides an excellent starting pattern but a package needs tests that are independent of DD models.

At minimum, the extracted project should test the same source fixture against:

```
CPython standard dataclasses
CPython running the MicroPython compatibility implementation
MicroPython v1.29.0
constrained MicroPython heap execution
```

The semantic corpus should include constructor errors, positional/keyword combinations, required fields, defaults, factories, mutable-factory isolation, equality, exact-class comparison, `__post_init__`, frozen writes/deletes, malformed manifest data, missing manifest entries, stale manifests, module renames and duplicate identities.

The project's initially advertised MicroPython baseline should be **v1.29.0**, because that is the constrained interpreter version DD actually qualified. Broader or older MicroPython compatibility should be claimed only after CI proves it. Likewise, the host project's minimum CPython version should be selected from an explicit test matrix rather than inferred from what the implementation happens to run on today.

This distinction prevents a successful application extraction from immediately inheriting an unbounded compatibility promise.

StrEnum and the smaller compatibility mechanisms

DD's `StrEnum` implementation is the clearest example of code that is **clever inside a closed application and wrong as a general library abstraction**.

At import time, the module saves the current `builtins.__build_class__`, installs a wrapper in its place, allows every subsequent Python class to be created through that wrapper, and then checks whether

the newly created class directly inherits from DD's `StrEnum`. If so, it converts uppercase string-valued class attributes into enum member objects and creates the lookup structures.

Within DD this has attractive properties. It permits fairly natural class syntax and solves a concrete absence without requiring widespread source rewrites.

As a public library, however, process-wide `__build_class__` interception causes several problems:

```
it affects class creation unrelated to the library
it must compose with any other tool that wraps __build_class__
import order can become behaviorally significant
debuggers/instrumentation/metaprogramming tools can observe the wrapper
uninstallation/restoration semantics become another API problem
multiple versions of the library could compete for global state
failure inside post-processing occurs after ordinary class construction
```

The current implementation even stores hook state globally on `builtins`, which demonstrates that the code already has to solve process-wide coordination rather than merely Enum behavior.

That is not a criticism of the DD engineering decision. The risk calculus changes when a controlled application's compatibility layer becomes a dependency loaded into arbitrary programs.

The current ecosystem also removes most of the reason to create a separate `StrEnum` project.

`micropython-lib` PR #980, opened in 2025 and updated as recently as August 25, 2026, proposes a substantial pure-Python Enum module with `Enum`, `IntEnum`, and `StrEnum`, plus tests. It remains an open draft rather than merged functionality, so DD cannot simply assume the problem is solved today.

A separate MicroPython-core proposal, PR #16842, was closed rather than merged. The discussion explicitly pointed toward `micropython-lib`, rather than putting a large pure-Python compatibility implementation into MicroPython core.

The clean decision is therefore:

Current DD `StrEnum`: KEEP INTERNAL.

General `StrEnum` ecosystem work: CONTRIBUTE UPSTREAM to `micropython-lib`, especially tests and behavior cases discovered by DD.

Useful DD contributions could include test cases for:

```
string behavior
construction by value
member identity
duplicate handling
```

```
__members__  
name/value access  
subclass restrictions  
serialization-facing behavior DD actually depends on
```

There is no reason for an upstream contribution to contain the `__build_class__` interception itself.

The same argument applies to DD's member reconstruction code. The current `_finish_enum` scans uppercase attributes, requires strings and unique values, replaces the attributes with constructed members, assigns `.name` and `.value`, stores a member map, and replaces `__new__` with value lookup. Those semantics belong inside an Enum implementation, not as a separate “enum reconstruction” utility.

The smaller compatibility modules mostly fail the extraction test for the opposite reason: they are **too little abstraction**, not too much.

The readonly mapping is a straightforward wrapper exposing item lookup, iteration and length while retaining a reference to the source mapping. It is a good local replacement for the behavior DD needs and a weak package idea.

`first_or_none` is quite literally the minimum implementation one would expect: iterate once, return the first value, otherwise return `None`. A public dependency would make this operation less portable rather than more portable.

The random adapter has more engineering substance. It delegates to an injected object's `randrange` when supplied, otherwise uses `random.randrange` where available and falls back to rejection sampling over `getrandbits`. That is a sound way to avoid biased `% stop` reduction, but the useful abstraction is DD's injectable randomness boundary, not a new random library.

The regex helper approximates `fullmatch` by calling `match` and checking that the returned match reaches the end of the input, including an accommodation for reduced match objects. Again, this is good compatibility glue and poor standalone product material.

The `collections.abc.Collection` fallback is potentially more dangerous to generalize. It turns `isinstance` into a structural test for `__len__`, `__iter__`, and `__contains__`. That is enough for DD's usage but is not a substitute for the full semantics, registration behavior, inheritance relationships, and companion ABCs users associate with `collections.abc`.

The typing overlay is even more explicitly a source-unblocking mechanism. `Protocol` becomes `object`; `runtime_checkable` becomes an identity decorator; `TypeVar` returns a trivial substitute; several subscriptable typing names are lightweight placeholders; and `TYPE_CHECKING` is false. That is exactly the right kind of overlay when runtime typing behavior is irrelevant. It should not be advertised as a general MicroPython `typing` implementation.

The `keyword` module is the exception among the tiny shims because it mirrors a concrete standard-library API and has essentially no application policy. DD's implementation is a keyword/soft-keyword table plus

`iskeyword()` and `issoftkeyword()`. Current `micropython-lib` master inspection did not reveal a merged `python-stdlib/keyword` package.

That makes it a reasonable **small upstream contribution**, provided its tables are tested against the actual MicroPython parser rather than copied blindly from a particular CPython release. It is not worth creating a separate PyPI repository.

The source rewrites are clearer still:

```
Counter removal
groupby removal
strict-zip removal
starred-tuple portability changes
portable direct string/ASCII validation
```

These represent an important portability lesson: **the best compatibility layer is frequently no compatibility layer**. When equally readable source can express the operation on both runtimes, reintroducing an abstraction as a package would reverse the improvement.

Qualification tooling

DD's qualification infrastructure is arguably the second most technically interesting output of the campaign, but it has a weaker product case than the dataclass work.

The current matrix is not merely “run tests under Python twice.” It defines a sequence of required stages including native CPython, CPython with the portability overlay forced, manifest freshness, a raw MicroPython probe, MicroPython gameplay/spec execution, and constrained-heap qualification. It launches subprocesses, records structured status, parses a designated JSON semantic-signature record from test output, and compares cross-runtime semantics.

That is a strong validation architecture because it answers several different questions:

Does ordinary CPython behavior still work?

Does the portability implementation itself work even when CPython would normally use richer standard-library features?

Can a raw MicroPython interpreter boot the application environment?

Does the same behavioral scenario produce equivalent semantic output?

At what constrained heap size does the runtime stop qualifying?

Those questions are reusable.

The present implementation is not yet a reusable framework. The orchestrator knows DD's named stages, exact test commands, manifest generator, probe path, MicroPython version/build policy, output markers, and pass/fail conventions. Extracting it now would force arbitrary DD decisions into a public configuration schema.

The **semantic signature** is the strongest generic concept. Rather than assuming identical textual output, each runtime emits a normalized JSON description of behavior that matters, and the host controller compares those results. A general pattern might look like:

```
runtime A → scenario → normalized JSON signature
runtime B → scenario → normalized JSON signature
                        |
                        ▼
                    semantic compare
```

That pattern is valuable well beyond games. A serialization package could compare encoded models; a parser could compare AST-like structures; a business-rules library could compare state transitions; an embedded application could compare deterministic command sequences.

What should be shared today is the **pattern and perhaps a tiny internal utility for extracting signatures**, not a public configuration framework.

The heap sweep is similarly well designed for its purpose. It launches a new MicroPython process for each heap limit, includes an unrestricted baseline, tracks phase markers, distinguishes ordinary failures from memory failure, records structured results, and can enforce a minimum qualifying heap. Using a fresh process per measurement avoids letting one heap trial contaminate the next.

Again, the generic idea is excellent:

```
fresh interpreter
→ fixed scenario
→ fixed heap ceiling
→ structured result
→ repeat across ceilings
→ find qualification floor
```

The present code still understands DD's bootstrap and probe output, and its default heap sequence and acceptance threshold are project policy.

A future external tool might eventually expose something closer to:

```
mpy-heap-sweep \
  --interpreter "micropython ..." \
  --command test_probe.py \
```

```
--heaps 16k,24k,32k,48k,64k,96k \  
--success-marker ...
```

but publishing that now would be premature. The command protocol, memory-setting mechanism, reporting schema, port compatibility, timeout behavior and interpretation of fragmentation all need experience outside DD before they should become public API.

The raw bootstrap/probe likewise belongs in DD because probes are meaningful only when they probe **specific invariants**. A universal “MicroPython probe” would rapidly accumulate every runtime peculiarity and turn into exactly the giant compatibility framework this audit should avoid.

The appropriate classification is therefore:

Qualification architecture: reusable pattern worth documenting.

Semantic signature extractor/comparator: potentially reusable utility, but extract only after another project uses it.

Heap sweep controller: promising seed of a tool, but **RESEARCH FURTHER** rather than package now.

DD probe/bootstrap and full matrix: KEEP INTERNAL.

A practical extraction criterion should be strict: **do not publish the qualification framework until one non-DD project can use the same core without the core learning anything about that project's test names, directories, output markers or domain objects.**

Ecosystem and user fit

The main ecosystem question is not whether alternatives exist. It is whether they solve the **same problem under the same source-compatibility constraints**.

For dataclasses, the answer is currently no.

MicroPython's documented philosophy is to reduce or omit some runtime processing associated with Python annotations because those facilities impose costs that constrained deployments often do not need.

³ This makes annotation-driven runtime frameworks structurally awkward even when the source syntax parses.

The current `micropython-lib` project is explicitly intended to provide Python-standard-library functionality and MicroPython-oriented packages outside the core runtime. ⁴ Inspection of its present tree found no merged `python-stdlib/dataclasses` package. A repository search likewise did not surface a current dedicated dataclasses implementation or active dataclasses pull request there.

`udataclasses` remains the most important current alternative examined in this audit. Its repository describes itself as a lightweight dataclass implementation intended for MicroPython and low-memory use.

Its visible commit history shows code/release work through January 2025, so it should not be dismissed as an ancient abandoned experiment, although the project has not shown the cadence of a heavily maintained compatibility layer in the inspected history.

Its documented semantic differences are exactly why DD's idea remains interesting. `udataclasses` uses public class variables rather than CPython's normal annotation-defined field model, expects initial field values, sorts fields, treats types as runtime-irrelevant, and differs in several details of initialization/frozen behavior.

The comparison is therefore:

`udataclasses` is attractive when the application is willing to author models according to `udataclasses` conventions.

DD's generated-manifest approach is attractive when the application wants CPython to remain the semantic authority and MicroPython to consume a precomputed projection of that authority.

Those are genuinely different users.

The likely dataclass-toolchain user is not “a Dungeon Drifters-like game developer.” It is:

a developer or library author maintaining one model layer across full CPython and constrained MicroPython, who wants ordinary CPython dataclass declarations to remain the source of truth and is willing to add a build-generation step rather than weaken those declarations for the runtime.

Examples include embedded control/configuration applications, protocol/model libraries, offline-capable devices, game/application engines embedding MicroPython, and packages that already generate deployment artifacts.

The strongest signal of fit is a project containing constructs such as:

```
@dataclass
class Message:
    sequence: int
    payload: bytes
    flags: int = 0
```

where host-side tests, IDEs, type checkers and ordinary CPython all benefit from standard source, but the deployed interpreter should not pay to retain rich annotation/reflection state.

This is a much clearer audience than “people who need MicroPython dataclasses.”

For Enum/StrEnum the ecosystem situation is almost the reverse. There **is** a substantial active upstream proposal. `micropython-lib` PR #980 currently targets `Enum`, `IntEnum`, and `StrEnum`. Consequently, a new DD-derived package would fragment effort precisely when a more standard home is already visible.

The MicroPython-core Enum proposal also provides useful architectural evidence. That proposal was closed, with discussion pointing pure-Python compatibility implementation toward `micropython-lib`. The correct ecosystem destination for this kind of stdlib compatibility is therefore much clearer than it is for DD's build-time dataclass metadata generator.

For tiny stdlib shims, the same principle applies. A `keyword` compatibility implementation can plausibly improve `micropython-lib`; an independent “Dungeon Drifters keyword compatibility” package has no defensible reason to exist.

No similar ecosystem argument supports publishing `first_or_none`, readonly mapping, random fallback, regex `fullmatch`, or structural `Collection` as individual packages.

One limitation of the ecosystem survey should be explicit. The primary-source research was sufficient to examine current MicroPython documentation, current `micropython-lib`, its active Enum work, and the current `udataclasses` repository. It did **not** produce a defensible exhaustive catalog of every historical PyPI mirror or abandoned dataclass experiment. I have therefore not treated unverified package-index mirrors as meaningful competitors. That gap does not materially change the main result because the strongest comparison is with a currently available implementation, `udataclasses`, and with the current official library repository.

Extraction sequence, repository shape, and what stays in DD

The extraction should begin by proving that the **dataclass manifest contract exists independently of DD**, not by moving DD files into a new repository and calling that a package.

The first step should be the host-side generator and schema.

```
DD generator
  ↓ remove DD paths and discovery policy
generic CPython discovery library
  ↓
versioned deterministic manifest
  ↓
toy package fixture proving no DD dependency
```

The acceptance condition is simple: a small unrelated package containing ordinary CPython dataclasses can generate and check a manifest without importing or naming any Dungeon Drifters code.

The second step is the runtime consumer.

```
versioned manifest
  ↓
zero-dependency runtime
  ↓
MicroPython compatibility facade
  ↓
same model fixture executes on CPython and MicroPython
```

This is where unsupported semantics must become explicit errors. A public library that quietly approximates `kw_only`, inheritance or rich `Field` introspection would be worse than one that says “unsupported.”

Only after that boundary is established should the package be integrated back into DD.

```
new generic package
  ↑
Dungeon Drifters becomes a consumer
```

That direction matters. If DD remains the implicit owner of hidden assumptions, the extraction has not really happened.

An initial repository could contain both distributions so schema and runtime changes remain coordinated:

```
repository
├── packages/
│   ├── host-generator/
│   └── micropython-runtime/
├── schema/
├── examples/
│   └── ordinary-package/
└── tests/
    ├── cpython-native/
    ├── cpython-forced-runtime/
    └── micropython/
```

Only split them into independent repositories later if release cadence or maintainership genuinely diverges.

The host package's one-sentence description could be:

Generate compact, versioned dataclass field metadata on CPython for applications that deploy the same model source to constrained Python runtimes.

The runtime package's description could be:

Consume ahead-of-time dataclass metadata to provide a deliberately small dataclass subset on MicroPython without runtime annotation reflection.

Neither package should mention DD in its API.

Neither package should contain:

```
Drifter
Battle
PlayerState
routes
save-game behavior
DD source roots
DD output paths
DD probe markers
DD heap thresholds
DD test names
DD runtime-signature schema
```

The generated artifact itself remains application-owned. The generic generator owns the **format and generation semantics**, not the content.

The integration model should be:

```
host development:
  CPython stdlib dataclasses
  +
  pip-installed generator

deployment build:
  generator emits manifest

MicroPython image:
  tiny runtime consumer
  +
  generated manifest
  +
  project source
```

The first release's explicit non-goals should include:

```
complete CPython dataclasses compatibility
runtime preservation of type annotations
general serialization
```

```
validation frameworks
schema migration
Pydantic-like behavior
ORM features
arbitrary code generation
Enum compatibility
general CPython-to-MicroPython portability
test orchestration
DD-specific bootstrapping
```

Keeping those out is critical. “We can generate metadata” is not a reason to become a schema framework.

The independent `keyword` contribution can proceed without waiting for the dataclass project because there is no dependency relationship. It should be proposed directly in `micropython-lib`, with parser-level tests for exactly which hard and soft keywords MicroPython recognizes. Current DD code is a good seed, not necessarily the final upstream source.

The StrEnum path should likewise be independent:

```
DD StrEnum test cases / requirements
↓
micropython-lib PR #980 or successor
↓
upstream implementation matures
↓
DD removes global __build_class__ workaround
```

The currently open upstream work makes that a better investment than cleaning DD's hook for publication.

Most of the portability campaign should remain inside Dungeon Drifters.

Specifically, DD should continue to own:

The application compatibility policy. Which stdlib gaps DD is willing to emulate is a product/runtime choice.

The generated DD manifest. It is a build artifact describing DD classes, not library source.

Session persistence boundaries. Those encode DD's architecture and which subsystems are allowed on the constrained runtime.

Randomness policy. The injected RNG and fallback behavior are part of deterministic gameplay/testing policy even though the small adapter is technically generic.

Iteration conveniences such as `first_or_none`.

Readonly mapping.

Regex and structural ABC shims, unless DD can later delete them because an upstream implementation becomes universally usable across its supported runtime matrix.

No-op typing compatibility, for the same reason.

Raw probe semantics, semantic signature contents, heap qualification threshold, and qualification stages. Those answer “does Dungeon Drifters qualify?” and should not be delegated to a generic package.

All source simplifications such as eliminating unsupported library conveniences. Those are improvements to DD source, not reusable artifacts.

A successful extraction would therefore actually make DD's portability directory **smaller**, not create a dependency for every historical workaround.

Risks, open questions, and final recommendations

The biggest risk in the dataclass extraction is **API ossification caused by using familiar standard-library names**.

The moment a public package exposes `dataclass`, `field`, or especially `__dataclass_fields__`, users will infer CPython behavior that DD itself never needed. The current implementation's `__dataclass_fields__` is only an ordered tuple of names, whereas CPython exposes substantially richer field objects. That discrepancy must either be removed from the public surface or documented and tested as an intentional reduced profile.

The second risk is **overgeneralizing from one successful application**. DD's dataclass usage is sufficiently regular that a compact implementation works. That does not prove the same contract handles arbitrary inheritance, descriptors, `InitVar`, `ClassVar`, generated classes, metaclasses, slots or every combination of standard decorator parameters. The right response is not to implement all of them. It is to reject them deterministically until real users justify support.

The third risk is **manifest compatibility**. Once generated artifacts are committed or distributed separately from the generator, schema changes become release-management events. The artifact therefore needs an explicit schema version from the first public release. The runtime should fail immediately on a schema version it does not understand.

The fourth is **generator execution behavior**. The current tool imports discovered application modules to ask real CPython dataclass machinery for the authoritative result. That is a powerful feature because it avoids reimplementing Python semantics, but it also executes trusted application import code. The package should state that clearly. DD's current special handling around modules containing bare `input()` calls should not become the basis of a fake sandbox.

The fifth is **module identity churn**. `(module, qualname)` is simple and good, but package relocation changes identity. This is preferable to fuzzy matching. A deployment build should fail because the manifest is stale rather than guessing that a similarly named class is “probably” the old class.

The sixth is **MicroPython version and port churn**. MicroPython intentionally implements a subset of Python and different ports can expose different practical constraints. ⁵ The package should separate “semantic support” from “qualified runtime combinations” and initially advertise only combinations its CI actually executes.

The seventh is **generated-code and generated-metadata burden**. Every build step creates user friction. The generator must therefore be deterministic, fast, invocable both as `build` and `check`, and produce useful diffs. DD already has the important deterministic/check behavior to build on.

The eighth is **invasive monkeypatching**, principally `StrEnum`. Process-wide `__build_class__` replacement should be treated as an application-local escape hatch, not an extraction foundation.

The ninth is **duplicating ecosystem functionality**. `dataclasses` must remain in the comparison and documentation so users understand why they would choose one model over the other. Enum duplication is even less defensible while a substantial `micropython-lib` implementation is under active review.

The tenth is ordinary **maintainer burden**. “MicroPython compatibility” can silently become an obligation to support many MicroPython releases, ports, hardware constraints, and partial stdlib combinations. A narrow dataclass artifact contract is maintainable. A general portability framework is not.

The open technical questions before a dataclass v0.1 are bounded and answerable:

Question	Recommended first-release answer
Manifest versioning	Required from day one
Deterministic generation	Already demonstrated; preserve it
Stale detection	Preserve <code>build</code> + <code>check</code> ; optional semantic/source fingerprint metadata
Identity	Exact module + qualname
Local classes	Reject
Inheritance	Reject or mark experimental until an explicit compatibility corpus exists
Defaults	Keep in class source
Default factories	Keep in class source through the runtime <code>field</code> marker
<code>frozen</code>	Support, but define exception semantics explicitly
<code>__post_init__</code>	Support

Question	Recommended first-release answer
Field metadata	Do not support initially
Runtime type annotations	Do not preserve
Imported/nested field types	Irrelevant unless runtime typing is later added
<code>repr</code>	Either implement normal default behavior or explicitly mark unsupported
Ordering/hash/slots/kw-only	Unsupported initially
<code>InitVar</code> /advanced field kinds	Unsupported initially
<code>__dataclass_fields__</code>	Do not expose a misleading CPython-like name unless its contract is intentionally implemented
Dependencies	Host: CPython stdlib only; runtime: zero third-party dependencies
MicroPython baseline	Initially claim v1.29.0, because that is the qualified DD target
Hardware-port claims	None until independently tested

The final recommendations are therefore:

Generated dataclass metadata: EXTRACT AFTER A SMALL GENERALIZATION PASS.

Where: a new narrow project, with a CPython host generator and zero-dependency constrained-runtime consumer as separable components.

For whom: developers sharing normal dataclass-authored models between CPython and MicroPython.

Problem solved: MicroPython need not retain or reconstruct annotation-driven dataclass field metadata at runtime.

Why: this is a real gap not addressed in the same way by the current `udataclasses` design or by merged `micropython-lib` functionality.

MicroPython dataclass runtime layer: EXTRACT AFTER A SMALL GENERALIZATION PASS.

Where: runtime half of the same project, potentially a future `micropython-lib` candidate only after its reduced compatibility contract is proven externally.

For whom: users of the generated manifest.

Problem solved: deterministic construction/equality/default/frozen/post-init behavior without runtime field discovery.

Why: useful only when paired with the manifest architecture; publishing the present DD module as a generic `dataclasses` replacement would overpromise.

Dataclass field reconstruction: EXTRACT AFTER A SMALL GENERALIZATION PASS as an internal runtime component, not its own package.

Where: inside the runtime package.

For whom: manifest users.

Problem solved: combine ahead-of-time field identity with runtime defaults/factories.

Why: coherent implementation mechanism, no standalone product case.

StrEnum: CONTRIBUTE UPSTREAM.

Where: `micropython-lib` Enum work, principally PR #980 or its successor.

For whom: all MicroPython users who need standard Enum-family behavior.

Problem solved: Enum/IntEnum/StrEnum compatibility without application-wide class-construction interception.

Why: active upstream work already targets the feature; DD's `__build_class__` hook is too invasive for public reuse.

DD Enum reconstruction/lookup implementation: DO NOT EXTRACT.

Where: nowhere as a standalone artifact. Keep it only as temporary DD compatibility code and use its behavior cases to improve upstream tests.

Why: it is implementation machinery for the preceding workaround, not a general abstraction.

Keyword compatibility: CONTRIBUTE UPSTREAM.

Where: preferably `micropython-lib`, subject to parser-compatible tests.

For whom: libraries sharing code that expects Python's small `keyword` API.

Problem solved: provide a tiny missing stdlib surface.

Why: it is exactly the kind of narrow standard-library compatibility unit that benefits more from the shared MicroPython library ecosystem than from another package.

Readonly mapping: KEEP INTERNAL.

Where: DD utility module.

Why: correct and useful, but too trivial and under-specified to justify ecosystem maintenance.

Regex fullmatch compatibility: KEEP INTERNAL.

Where: DD compatibility helper.

Why: one-function adaptation with no meaningful package boundary.

`collections.abc` compatibility: KEEP INTERNAL.

Where: DD compatibility helper.

Why: the current structural approximation is appropriate when DD controls its uses and too semantically loose to advertise as general ABC compatibility.

Typing / Protocol compatibility: KEEP INTERNAL.

Where: DD's constrained-runtime overlay.

Why: these objects intentionally provide import/syntax survivability rather than a general typing runtime.

Portable ASCII validation: DO NOT EXTRACT.

Where: ordinary portable source.

Why: the source rewrite is the solution.

Counter removal: DO NOT EXTRACT.

Where: ordinary DD code.

Why: removing a dependency created no independent reusable abstraction.

Starred-tuple portability changes: DO NOT EXTRACT.

Where: source code.

Why: syntax portability is better resolved at the source.

Strict-zip removal: DO NOT EXTRACT.

Where: source code.

Why: same reason.

Session persistence import boundary: KEEP INTERNAL.

Where: DD architecture.

For whom: DD runtime and host subsystems.

Problem solved: keep host-only persistence concerns from contaminating the constrained gameplay dependency graph.

Why: it is a good architectural boundary, not a reusable software product.

`groupby` **removal: DO NOT EXTRACT.**

Where: source code.

Why: direct iteration is the portable implementation.

`first_or_none` **: KEEP INTERNAL.**

Where: tiny DD iteration helper.

Why: a dependency for a one-loop function would be counterproductive.

Portable random adapter: KEEP INTERNAL.

Where: DD randomness boundary.

For whom: DD code needing injectable deterministic RNG behavior across runtimes.

Why: the implementation is technically good, including unbiased `getrandbits` rejection sampling, but too narrow to sustain a separate library.

Raw probe/bootstrap: KEEP INTERNAL.

Where: DD qualification tests.

Why: probes become useful through the DD-specific invariants they test.

Cross-runtime semantic qualification: RESEARCH FURTHER.

Where: currently DD; potentially a future small host-side test utility.

For whom: cross-CPython/MicroPython libraries that need behavioral equivalence rather than merely successful execution.

Problem solved: compare normalized semantic outcomes across interpreter configurations.

Why not now: the existing matrix still encodes DD's stage graph and output protocol. A second independent adopter should define the abstraction before API publication.

Constrained-heap sweep: RESEARCH FURTHER.

Where: currently DD; possibly a future standalone host CLI if reused elsewhere.

For whom: MicroPython applications that need reproducible minimum-heap qualification.

Problem solved: run equivalent workloads under isolated heap ceilings and produce structured failure information.

Why not now: the algorithm is reusable, but the live tool is still coupled to DD's bootstrap, markers, policy and thresholds.

The extraction sequence that follows from those judgments is:

```
generalize dataclass manifest schema and host generator
    ↓
prove it against a non-DD fixture
    ↓
extract the zero-dependency runtime consumer
    ↓
make DD consume the extracted project
    ↓
evaluate whether the runtime half belongs in micropython-lib

independently:
contribute keyword compatibility upstream

independently:
feed DD Enum requirements/tests into micropython-lib Enum/StrEnum work

later, only after a second real user exists:
reconsider semantic-signature and heap-sweep tooling
```

That sequence preserves the central lesson of the MPY0-MPY20 campaign: **portability work is reusable when it captures a stable problem boundary, not merely when its source code can be moved to another repository.**

On that standard, the generated dataclass metadata system passes. The current `StrEnum` implementation does not, although the requirements behind it are valuable upstream. The qualification tooling may eventually pass after another independent use case exposes the right generic seam. Most of the remaining mechanisms are better evidence of good portability engineering than candidates for new software products.

1 2 3 <https://docs.micropython.org/en/v1.29.0/genrst/syntax.html>

<https://docs.micropython.org/en/v1.29.0/genrst/syntax.html>

4 <https://github.com/micropython/micropython-lib>

<https://github.com/micropython/micropython-lib>

5 <https://github.com/micropython/micropython>

<https://github.com/micropython/micropython>